

JIT-AGENT: 通过即时智能体支架演化扩展支架智能

JIT-Agent 团队

摘要

智能体能力并非仅由模型决定。智能体支架涵盖记忆管理、规划策略、动作协议以及工具/技能编排，其作用可能压过底层基础模型本身的贡献。然而，支架设计仍依赖人工、针对特定任务，因而从根本上难以扩展。我们提出 JIT-Agent：一种支架智能模型，经过训练后可为任意现成的智能体大语言模型即时合成适应任务的智能体支架。我们将智能体支架形式化为一种可组合、可由机器生成，并受固定四模块协议约束的工件；随后训练 JIT-Agent 针对手头任务定制支架，为稳定可靠的执行修复支架，并从不断扩充的既往支架配置档案中提炼性能信号以实现自我演化。在 JIT-Agent 作为支架助手的情况下，DeepSeek-V4-Flash 在 DeepSearchQA 和 OdysseyBench 上分别超过 GPT-5.6 9.1 和 4.3 分，而本已强大的 GLM-5.2 最多还能提升 20.2 分。在受控评测中，JIT-Agent 生成的支架在性能上可与 OpenCode、Claude Code 等成熟智能体运行时竞争，并能持续改善 DeepSeek V4、Mimo-V2.5 和 Qwen3.6 各种规模的模型。就我们所知，JIT-Agent 是首个专为即时支架生成而构建的模型：它确立了支架智能这一与模型扩展正交、可训练、可迁移且能持续积累的智能体能力维度。

🔗 代码: <https://github.com/bingreeky/JIT>

1 引言

智能体支架至关重要。大语言模型智能体的能力由两个紧密耦合的因素共同决定：产生推理与动作的**基础模型**，以及将该模型置于闭环执行环境中的**智能体支架** (Meng et al., 2026b; Zhou et al., 2026; Ning et al., 2026)。支架决定保留哪些历史、如何形成局部意图、暴露哪些工具或技能、如何执行动作，以及何时触发验证或恢复 (Meng et al., 2026b; Zhou et al., 2026)。强模型若置于错误的记忆、规划器或动作协议之后也可能失败；反过来，强支架也只有模型能够理解并遵循它时才能释放能力 (Lee et al., 2026c; Lin et al., 2026)。因此，智能体智能并非模型权重独有的属性，而是模型-支架组合的属性 (Lee et al., 2026c; Zhou et al., 2026)。由此，支架设计不是实现细节，而是决定智能体性能的一阶因素 (Meng et al., 2026b; Lin et al., 2026)。

预先 (AOT) 支架。 上述观察催生了越来越多关于测试时支架优化的研究 (Lee et al., 2026c; Lin et al., 2026)。近期系统从轨迹和反馈中优化支架代码、提示、工具、记忆、技能或控制策略，在编码、终端使用、Web 交互和通用智能体领域取得了显著改进 (Lee et al., 2026c; Lin et al., 2026)。尽管这些方法彼此不同，许多方法都共享一个预先 (Ahead-of-Time, AOT) 假设：将支架视为在经验流上持续优化的耐久工件，并期望所得工件能够泛化至未来的任务、领域或模型版本 (Lee et al., 2026c; Lin et al., 2026)。当部署分布稳定且同质时，这是一个强大的范式 (Lin et al., 2026; Zhang et al., 2025)。然而，它仍要求优化环在看到每个未来问题的确切结构之前，预先编译出一个普遍有用的支架 (Lee et al., 2026c)。

即时 (JIT) 支架。 我们转而采用一种**即时 (Just-in-Time, JIT) 的支架构建观**。不同任务显然需要不同的支架先验。广域搜索任务可能受益于并行证据探索 (Qin et al., 2025)；终端任务通常更适合精简、串行的 ReAct 环 (Yao et al., 2023a; Merrill et al., 2026)；深度研究任务需要围绕检索证据维护工作记忆 (Hu et al., 2025)；而 NL2Repo 风格的编码任务则天然由文件系统居中协调，文件系统存放补丁、测试、轨迹和仓库状态 (Yang et al., 2024; Ning et al., 2026)。简言之，合适的支架不仅取决于领域，还取决于具体实例 (Meng et al., 2026b; Zhou et al., 2026)。面对这些异质需求，优化单个 AOT 支架十分繁琐：系统必须搜索庞大的设计空间、积累足够多的轨迹，然后期待所得支架恰好适配下一项任务 (Lee et al., 2026c; Lin et al., 2026)。我们由此追问另一种可能性是否可行：**模型即支架 (Model-as-a-Harness)**，即由训练过的元智能体现场生成任务专用支架，再由任意智能体大语言模型在该支架下执行。

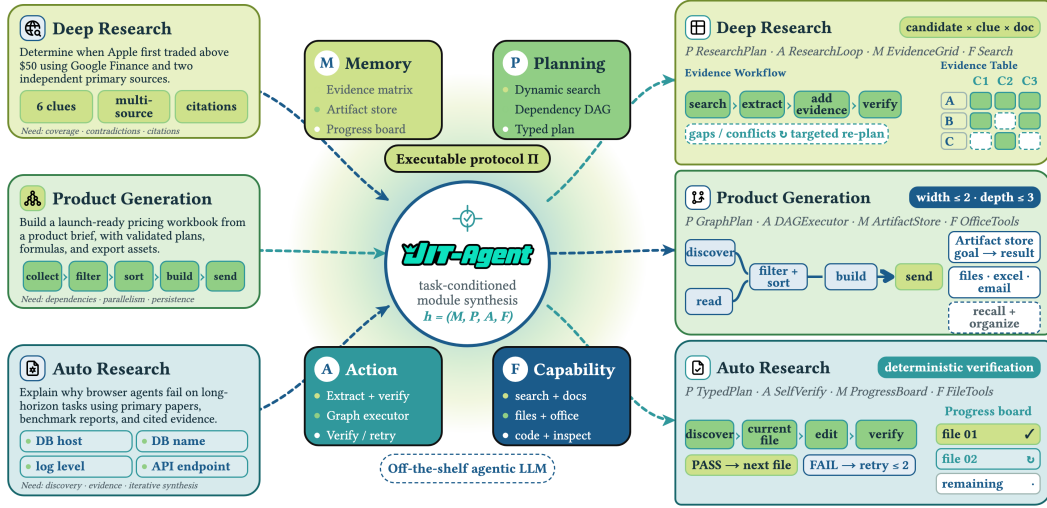


图 1: JIT-Agent 概览。给定一项任务，JIT-Agent 通过实例化（而非简单拼接）记忆、规划、动作和能力四个模块，构造面向该问题的智能体支架。因此，不同的任务结构会诱导出不同的可执行协议与状态组织方式；图中分别展示了面向深度研究、产品生成和自主研究的专用支架。

JIT-Agent。 我们提出 JIT-Agent，一种用于即时支架生成与演化的紧凑元智能体。在推理时，JIT-Agent 接收任务规范、协议、可执行工具/技能注册表，以及检索得到的一小段既往支架上下文，并输出为手头任务量身定制的可执行支架。所得支架包装一个现成的智能体大语言模型并负责执行。随着反馈与轨迹不断积累，JIT-Agent 可修订支架并更新支架档案，从而在生成器本身保持固定的情况下实现测试时支架演化。训练这样的系统面临三项挑战：**① 适应性**，即如何让生成的支架匹配任务；**② 可靠性**，即如何保证行为可执行，并在合成失败时恢复 (Pan et al., 2026; Ning et al., 2026)；以及 **③ 可演化性**，即如何将执行反馈转化为未来更强的支架 (Lee et al., 2026c; Lin et al., 2026)。这三项要求共同定义了我们所说的**支架智能**：

支架智能

支架智能是构建并完善模型赖以行动的运行脚手架的能力。它有三个决定性属性：**适应性**，使支架匹配任务与骨干模型；**可靠性**，产生可执行行为，并在合成失败时恢复；**可演化性**，将执行反馈转化为未来更强的支架。固定支架内的能力抵达边界之处，正是支架智能开始之处。

JIT-Agent 在协议诱导的支架空间上采用三阶段训练方案来实现支架智能 (Meng et al., 2026b; Zhou et al., 2026)。首先，我们将智能体支架分解为由记忆、规划、动作和能力编排模块组成的四元组，并构建 HARNESSTORY：在这一公共接口下汇集代表性脚手架的统一代码库 (Hu et al., 2025; Erdogan et al., 2025a; Yao et al., 2023a; Xu & Yan, 2026)。遵循“代码承载支架”的观点 (Ning et al., 2026; Pan et al., 2026)，这一表示使支架生成成为可能：JIT-Agent 生成结构化的可执行模块，而不是不受约束的智能体程序。随后，**① 阶段 I** 从教师生成的协议合规样例中学习任务条件化定制，使生成器在共享设计空间中具备适应性。其次，**② 阶段 II** 将失败的生成结果转化为有界修复轨迹，把编译器错误、接口不匹配和运行时故障变成可靠恢复的监督信号。第三，**③ 阶段 III** 引入演化式组解耦策略优化 (Evolutionary Group-Decoupled Policy Optimization, Evo-GDPO)，分别归一化奖励、延迟和成本，同时优化 JIT-Agent，使其提出能够超越当前档案前沿的支架。这一步把测试时支架改进从外部搜索启发式转变为训练所得能力，从而提供可演化性。通过这条流水线，我们训练得到 **JIT-Agent-27B**，一种能够定制、修复和演化任务专用支架的**支架智能模型**，将支架工程从 AOT 工件转向原生 JIT 合成。

实验发现。 在九个基准上，JIT-Agent 持续增强其底层骨干模型：DeepSeek-V4-Flash 在 DeepSearchQA 和 OdysseyBench 上分别超过 GPT-5.6 9.1 分和 4.3 分，而 GLM-5.2 最多提高 20.2 分。生成的支架也能与成熟运行时竞争，并可迁移至 DeepSeek V4、Mimo-V2.5 和 Qwen3.6。

贡献。 (I) 我们将模型即支架形式化, 并把支架智能定义为通过学习获得的、构建、修复和演化智能体运行脚手架的能力。(II) 我们以 JIT-Agent 实例化该范式, 将四模块可执行协议、HARNESSFACTORY 与定制-修复-演化三阶段流水线相结合; (III) 我们验证了 JIT 生成的支架能够改善多种骨干模型, 同时与强大的生产级支架保持竞争力。

2 相关工作

支架工程与模块化。 支架已扩展到多种形态, 包括 Claude Code、Codex、Hermes 和 OpenClaw 等编码与通用运行时 (Anthropics, 2025; OpenAI, 2025; Nous Research, 2026; OpenClaw Contributors, 2026)、具身系统 (Zhang et al., 2026c; Lee et al., 2026b)、搜索智能体 (Apodex Team, 2026; Qin et al., 2025), 以及以记忆为中心的系统 (Chhikara et al., 2025; Hu et al., 2026a; Zhang et al., 2025)。尽管各领域的实现不同, 它们都包含一个支配记忆、规划与控制、动作验证以及工具/技能编排的运行层, 使能力成为模型-支架组合而非模型单独的属性 (Meng et al., 2026b; Zhou et al., 2026; Pan et al., 2026)。模块化表示通过显式接口暴露这些选择, 使各组件能够独立替换与优化 (Chen et al., 2026b; Ning et al., 2026)。因此, 我们采用紧凑的可执行分解 $\mathbf{h} = (\mathbf{M}, \mathbf{P}, \mathbf{A}, \mathbf{F})$, 分别表示记忆、规划、动作和能力编排。

支架优化。 有了显式接口, 支架构建便从固定的工程选择转变为针对可执行工件的优化问题 (Khatab et al., 2023; Yuksekgonul et al., 2024; Hu et al., 2024b; Shang et al., 2024)。近期方法把优化范围从提示和工作流程图扩展到完整支架代码, 通常采用“诊断-提议-验证”循环: 轨迹指导编辑, 而验证决定是否保留候选 (Zhang et al., 2024; Lee et al., 2026c; Lin et al., 2026; Chen et al., 2026b)。递归式、层次式和在线变体改进了这一循环, 但仍主要依据累积经验优化持久工件, 因此相对于新的任务实例仍属于预先方法 (Lee et al., 2026a; Zhou, 2026; Nie et al., 2026; Shao et al., 2026)。JIT-Agent 则把搜索摊销到训练所得的生成器中, 由其即时构建任务条件化支架; 表 4 总结了这一区别。

3 统一支架代码库

3.1 模块化支架设计空间

智能体支架是将基础模型转化为闭环智能体的运行层 (Meng et al., 2026a; Zhou et al., 2026): 它决定保留既往交互中的哪些内容 (Hu et al., 2026b)、如何形成中间指令 (Cao et al., 2025)、每个阶段暴露哪些外部能力 (Xu & Yan, 2026; Yang et al., 2026), 以及如何推进控制。因此, 支架生成定义在程序而非不受约束的文本之上。我们区分原始生成空间 $\mathcal{G}$ 、语法有效的支架 $\mathcal{H}^{\text{syn}}$ 、符合协议的支架 $\mathcal{H}_{\Pi}$ 及其中的可执行子集 $\mathcal{H}_{\Pi}^{\text{exec}}$, 它们满足 $\mathcal{G} \supseteq \mathcal{H}^{\text{syn}} \supseteq \mathcal{H}_{\Pi} \supseteq \mathcal{H}_{\Pi}^{\text{exec}}$ 。固定协议 Π 规定模块模式与接口、模块生命周期、验证规则和共享执行语义。它消除了语言和运行时中的偶然差异, 同时保留区分现有支架的主要运行选择 (Wu et al., 2023; Microsoft, 2024; LangGraph, 2025; Luo et al., 2025)。JIT-Agent 通过四个可互操作模块显式表达这些选择: 如何压缩历史、如何形成局部意图、如何编排工具与技能, 以及如何推进控制。令 τ 为一项任务, π_{ψ} 为冻结的骨干执行器, $\mathcal{C}_{\tau}$ 为该任务可用的能力注册表 (例如工具、API 和技能)。使用 π_{ψ} 运行支架 $\mathbf{h}$ 会产生闭环轨迹

$$\xi \sim \text{Rollout}(\tau, \pi_{\psi}, \mathbf{h}, \mathcal{C}_{\tau}; \Pi) = (\mathbf{s}_1, e_1, \mathbf{o}_1, \dots, \mathbf{s}_T, e_T, \mathbf{o}_T), \quad (1)$$

其中, $\mathbf{s}_t \in \mathcal{S}$ 是所维护的控制器状态, $e_t \in \mathcal{A} = \mathcal{U} \sqcup \mathcal{Y}$ 是来自 $\mathcal{U}$ 的可执行调用或来自 $\mathcal{Y}$ 的终止输出, $\mathbf{o}_t \in \mathcal{O}$ 是所得观察, 而 T 是由协议或预算限制的停止时刻。我们的关键假设是, 每个 $\mathbf{h} \in \mathcal{H}_{\Pi}$ 都允许如下分解:

$$\mathbf{h} = (\mathbf{M}, \mathbf{P}, \mathbf{A}, \mathbf{F}) \in \mathfrak{M} \times \mathfrak{P} \times \mathfrak{A} \times \mathfrak{F}, \quad (2)$$

其中, $\mathbf{M}$ 、 $\mathbf{P}$ 、 $\mathbf{A}$ 和 $\mathbf{F}$ 分别表示记忆、规划、动作和能力编排模块, 而 $\mathfrak{M}$ 、 $\mathfrak{P}$ 、 $\mathfrak{A}$ 和 $\mathfrak{F}$ 是各自与协议兼容的实现空间。该元组遵循上述概念分解; 在运行时, 其依赖顺序为 $\mathbf{M} \rightarrow \mathbf{P} \rightarrow \mathbf{F} \rightarrow \mathbf{A}$ 。所有模块都面向同一个冻结的 π_{ψ} 运行, 下文省略这种共同的运行时依赖。该协议同时维护不可变事件历史 $\xi_{<t}$ 和可变控制器状态 $\mathbf{s}_t$, 二者按如下方式交互:

$$\begin{aligned} \mathbf{v}_t &= \mathbf{M}(\xi_{<t}, \mathbf{s}_t) \in \mathcal{V}, & \text{历史} &\rightarrow \text{视图}, \\ \mathbf{d}_t &= \mathbf{P}(\tau, \mathbf{s}_t, \mathbf{v}_t) \in \mathcal{D}_{\text{dir}}, & \text{视图} &\rightarrow \text{局部指令}, \\ \mathcal{C}_t &= \mathbf{F}(\mathcal{C}_{\tau}, \mathbf{s}_t, \mathbf{v}_t, \mathbf{d}_t) \subseteq \mathcal{C}_{\tau}, & \text{以指令为条件的能力编排}, \\ (\mathbf{s}_{t+1}, e_t) &= \mathbf{A}(\mathbf{s}_t, \tau, \mathbf{v}_t, \mathbf{d}_t, \mathcal{C}_t) \in \mathcal{S} \times \mathcal{A}, & \text{控制更新与动作发出}, \end{aligned} \quad (3)$$

表 1: 种子库 $\mathcal{B}_0$: 13 个手工编写、实例化四模块协议 Π 的支架。每一行都是完整支架；各列依次对应记忆、规划、动作和能力编排。

支架	♣ 记忆 $\mathbf{M} \in \mathfrak{M}$	♦ 规划 $\mathbf{P} \in \mathfrak{P}$	♥ 动作 $\mathbf{A} \in \mathfrak{A}$	♠ 能力编排 $\mathbf{F} \in \mathfrak{F}$
REACT [69]	完整历史	无显式规划器	ReAct	完整注册表
PLAN-AND-EXECUTE [14]	完整历史	线性路线图	ReAct	完整注册表
RESUM [63]	ReSum 记忆	无显式规划器	ReAct	完整注册表
FLASH-SEARCHER [51]	完整历史	DAG 规划	ReAct	完整注册表
GAM [65]	GAM 检索	DAG	ReAct	完整注册表
MEMOBRAIN [50]	推理图	无显式规划器	标记引导执行	完整注册表
AGGAGENT [29]	隔离的回合历史	无显式规划器	多回合聚合	完整注册表
OAGENT [80]	协调器历史	无显式规划器	集成投票	完整注册表
AGENTFOLD [70]	AgentFold 记忆	DAG	ReActFold	完整注册表
HiAGENT [20]	层次记忆	无显式规划器	ReAct	完整注册表
DEEPAGENT [30]	三层记忆	无显式规划器	标记引导执行	工具搜索
ROMA [1]	上下文隔离	原子化器 + DAG	递归执行	完整注册表
AORCHESTRA [53]	上下文隔离	原子化器 + DAG	递归执行	智能体委派

其中, $\mathcal{V}$ 和 $\mathcal{D}_{\text{dir}}$ 分别是视图空间和指令空间。注册表 $\mathcal{C}_\tau$ 包含可调用工具 (例如 bash 工具、API 和 MCP (Sarkar & Sarkar, 2025)) 以及更高层的智能体技能 (Xu & Yan, 2026; ClaudeCode, 2025); 没有显式规划器的支架仍可通过 $\mathbf{P}_0$ 返回的空指令 $\mathbf{d}_0 \in \mathcal{D}_{\text{dir}}$ 保持类型一致。因此, $\mathbf{M}$ 从已实现历史中构建视图, $\mathbf{P}$ 形成局部指令, $\mathbf{F}$ 激活相关能力, 而 $\mathbf{A}$ 更新控制器状态并发出下一动作。共享内核 $\text{Exec} : \mathcal{U} \times 2^{\mathcal{C}_\tau} \rightarrow \mathcal{O}$ 在 $e_t \in \mathcal{U}$ 时令 $\mathbf{o}_t = \text{Exec}(e_t; \mathcal{C}_t)$, 在 $e_t \in \mathcal{V}$ 时令 $\mathbf{o}_t = \perp$, 其中 $\perp \in \mathcal{O}$ 是终止空观察; 随后它更新 $\xi_{\leq t} = \xi_{< t} \oplus (\mathbf{s}_t, e_t, \mathbf{o}_t)$, $\oplus$ 表示追加事件。执行从 $\xi_{< 1} = \emptyset$ 和协议规定的 $\mathbf{s}_1 \in \mathcal{S}$ 开始, 在 $e_t \in \mathcal{V}$ 时终止, 并得到 $y = e_T \in \mathcal{V}$ 。这一分解把原本异质的程序映射到 $\mathfrak{M} \times \mathfrak{P} \times \mathfrak{A} \times \mathfrak{F}$ 中可比较的坐标: 规范 ReAct 使用完整历史记录、无显式规划器、ReAct 环和完整能力注册表; Codex、OpenCode 等工程化变体加入紧凑记忆与基于待办事项的规划 (OpenAI, 2025; OpenCode-AI, 2025); ROMA、AOrchestra 和 RLM 等递归系统则实例化子问题记忆、分解、递归动作与路由能力 (Alzu'bi et al., 2026; Ruan et al., 2026; Zhang et al., 2026a)。基于这一模块化设计空间, 我们接下来引入 HARNESSTORY; 它用于检验 $\mathcal{H}_\Pi$ 的表达能力, 并为 JIT-Agent 的元支架设计提供多样化素材。

3.2 HARNESSTORY

在共享协议 Π 和公共内核下, HARNESSTORY 重新实现了 13 种代表性支架, 覆盖记忆、规划、聚合、层次控制和递归执行; 表 1 给出了完整清单。这些实现构成大小为 $K_0 = 13$ 、与协议兼容的种子库 $\mathcal{B}_0$ 。该库具有双重作用: 种子为阶段 I 的合成提供锚点, 而后续状态则提供评估新设计所依据的既往种群。经过 n 轮演化后, $\mathcal{B}_0$ 扩展为 $\mathcal{B}_n \supseteq \mathcal{B}_0$; 其中每个条目都把保留的支架与其任务以及观测到的奖励、延迟和成本关联起来。节 4.1 对这一演化过程作了形式化描述。

4 方法

4.1 训练流水线

训练遵循 JIT-Agent 的推理时生命周期: 先合成任务条件化支架, 再恢复不稳定的生成结果, 最后依据更强的档案状态在线改进。令 $\tau \sim \mathcal{D}_{\text{task}}$ 表示训练任务, $\mathcal{C}_\tau$ 表示其能力注册表, $\mathcal{E}_\tau$ 表示来自支架库的一小段参考上下文。生成上下文为 $\mathbf{c}_\tau = (\tau, \Pi, \mathcal{C}_\tau, \mathcal{E}_\tau)$, p_θ 据此生成支架 $\mathbf{h} \in \mathcal{G}$ 。冻结的执行器 $\pi_\psi \sim \mathcal{M}$ 评估任务奖励、延迟和货币成本, 而 $\text{Valid}_\Pi(\mathbf{h}; \tau, \pi_\psi, \mathcal{C}_\tau) \in \{0, 1\}$ 检查协议与执行有效性, 并在失败时返回结构化诊断。三个阶段分别通过离线监督、修复轨迹学习和在线策略改进来优化该生成器。

阶段 I: 数据准备。 阶段 I 使用一个冻结且更强的教师模型 q_ϕ , 在固定四模块协议下合成适应任务的支架。对每项任务, $\mathcal{E}_\tau$ 包含从任务类型匹配的子集 $\mathcal{B}_0^{(d(\tau))}$ 中采样的三个脚手架, 其中 $d(\tau)$ 表示任务类型。教师接收由此形成的上下文 $\mathbf{c}_\tau$; 只有通过协议验证和执行检查的生成结果才会保留。训练任务来自已有智能体基准和环境 (Wu et al., 2025; Shi et al., 2025; Bai et al., 2026; Song et al., 2026), 并辅以合成任务。

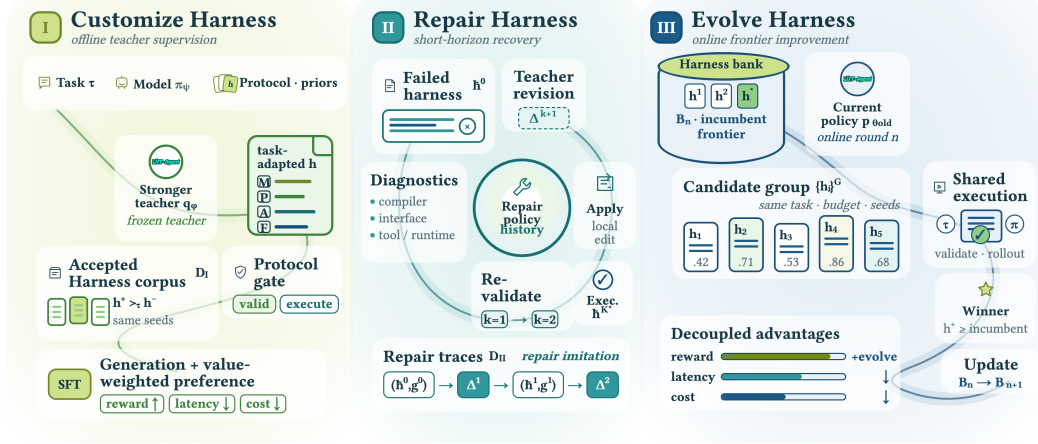


图 2: JIT-Agent 的训练流水线。阶段 I 学习任务条件化的支架定制。阶段 II 将失败支架与执行诊断转化为有界修复轨迹。阶段 III 把候选支架与当前支架库比较，通过优化解耦的奖励、延迟和成本优势开展在线演化，并保留能够推进前沿的设计。

阶段 I: 训练设置。 我们首先对被接受的教师生成结果进行标准监督微调，训练 JIT-Agent 将任务及其参考上下文直接映射为符合协议的支架。由于可执行支架在质量和效率上仍可能相差很大，我们随后在相同骨干模型与评估种子下比较候选。只有当奖励提高、两个效率维度均不退化，且至少一个效率增益为严格增益时，才保留该偏好：

$$\mathbf{h}^+ \succ_{\tau} \mathbf{h}^- \iff r^+ > r^- \wedge \ell^+ \leq \ell^- \wedge \kappa^+ \leq \kappa^- \wedge (\ell^+ < \ell^- \vee \kappa^+ < \kappa^-), \quad (4)$$

$$\Delta_{\text{val}}(\tau; \mathbf{h}^+, \mathbf{h}^-) = \alpha_r(r^+ - r^-) + \alpha_{\ell}[\ell^- - \ell^+]_+ + \alpha_{\kappa}[\kappa^- - \kappa^+]_+,$$

其中， $(r^{\pm}, \ell^{\pm}, \kappa^{\pm})$ 是 $(\mathbf{h}^+, \mathbf{h}^-)$ 的重复回合平均值， $[x]_+ = \max(x, 0)$ ， $\alpha_r, \alpha_{\ell}, \alpha_{\kappa} \geq 0$ 分别对奖励、延迟和成本差距加权。所得样本对用于监督一个标准 DPO 目标，该目标由 Δ_{val} 加权，并锚定到冻结的阶段 I SFT 检查点。与生成模仿结合后，该目标在保留协议有效结构的同时，更偏好效果和效率都更高的支架。

阶段 II: 数据准备。 阶段 I 以协议合规为优化目标，但部分生成支架仍无法通过静态或运行时验证。阶段 II 不丢弃这些样本，而是将每个失败支架 $\tilde{\mathbf{h}}^{(0)}$ 与结构化诊断 $\mathbf{g}^{(0)}$ 配对；诊断记录编译器错误、接口不匹配、工具调用失败或运行时异常。在第 k 轮修复中，教师从补丁空间 $\mathcal{P}$ 提出结构化修订 $\Delta^{(k+1)}$ ；Apply 确定性地更新支架，随后验证产生下一份诊断。我们只保留在两轮内变得可执行的轨迹：

$$\tilde{\mathbf{h}}^{(k+1)} = \text{Apply}(\tilde{\mathbf{h}}^{(k)}, \Delta^{(k+1)}), \quad k \in \{0, 1\}, \quad (5)$$

$$K^* = \min \left\{ k \in \{1, 2\} : \text{Valid}_{\Pi}(\tilde{\mathbf{h}}^{(k)}; \tau, \pi_{\psi}, \mathcal{C}_{\tau}) = 1 \right\}.$$

只有当 K^* 存在时才构造监督样本，使语料集中于现实中可局部恢复的失败，而非需要彻底重做的情形。

阶段 II: 训练设置。 阶段 II 通过教师强制模仿成功修订 $\{\Delta^{*(k+1)}\}_{k=0}^{K^*-1}$ 来训练 JIT-Agent。每次修订都以 $\mathbf{c}_{\tau}$ 和此前完整的支架-诊断历史 $\{(\tilde{\mathbf{h}}^{(j)}, \mathbf{g}^{(j)})\}_{j=0}^k$ 为条件。由于 $K^* \leq 2$ ，模型学习的是部署所需的短视野机制：利用执行反馈作出少量但高杠杆的修订，恢复符合协议的执行。

阶段 III: 数据准备。 阶段 III 将测试时支架演化视为可训练能力：JIT-Agent 学会提出超越既往设计的支架，而这些支架随后会成为后续任务更强的参考。受 (Liu et al., 2026a) 启发，我们将所得在线目标称为**演化式组解耦策略优化** (Evolutionary Group-Decoupled Policy Optimization, Evo-GDPO)。在第 n 轮在线训练中，我们采样任务 τ ，从当前支架库 $\mathcal{B}_n$ 检索既往高质量设计 $\mathcal{E}_{\tau, n}$ ，并形成 $\mathbf{c}_{\tau, n} = (\tau, \Pi, \mathcal{C}_{\tau}, \mathcal{E}_{\tau, n})$ 。模型从当前回合策略中独立采样 $G > 1$ 个候选；候选与检索到的设计在相同冻结执行器 $\pi_{\psi} \sim \mathcal{M}$ 、预算和种子下评估。检索支架中奖励最高者作为当前支架；若奖励相同，则先选择延迟较低者，再选择成本较低者，由此得到当前统计量 $(b_r, b_{\ell}, b_{\kappa})$ 。

阶段 III：训练设置。 每个候选都在执行前接受验证；经有界修复后仍无效的候选会获得最低任务奖励，同时其修复延迟和成本仍计入效率测量。对可执行候选 i ，令 r_i 、 ℓ_i 和 $\bar{\kappa}_i$ 分别表示重复回合中的奖励、平均延迟和平均货币成本。奖励通道优先；只有保持当前奖励时，效率增益才会激活：

$$R_i^{\text{rew}} = r_i + \lambda_{\text{evo}} [r_i - b_r]_+, R_i^{\text{lat}} = \mathbb{I}[r_i \geq b_r] [b_\ell - \bar{\ell}_i]_+, R_i^{\text{cost}} = \mathbb{I}[r_i \geq b_r] [b_\kappa - \bar{\kappa}_i]_+, \quad (6)$$

其中， $\lambda_{\text{evo}} \geq 0$ 控制改进奖励， $\mathbb{I}[\cdot]$ 是指示函数。Evo-GDPO 在聚合前对每个候选组内的三个通道分别归一化，防止任一通道因数值尺度而压倒其他通道：

$$A_i^m = \frac{R_i^m - \text{mean}(\{R_j^m\}_{j=1}^G)}{\text{std}(\{R_j^m\}_{j=1}^G) + \varepsilon_{\text{num}}}, \quad m \in \{\text{rew}, \text{lat}, \text{cost}\}, \quad (7)$$

$$A_i^\Sigma = w_{\text{rew}} A_i^{\text{rew}} + w_{\text{lat}} A_i^{\text{lat}} + w_{\text{cost}} A_i^{\text{cost}}, \quad w_{\text{rew}} > w_{\text{lat}} + w_{\text{cost}}.$$

这些非负权重之和为一，且上述不等式使任务奖励占主导； $\varepsilon_{\text{num}} > 0$ 用于稳定归一化。我们还在训练批次内归一化 A_i^Σ ，并将其用于标准裁剪 PPO 目标，同时加入相对于冻结阶段 II 检查点的词元级 KL 惩罚。因此，与标准组相对优化不同，Evo-GDPO 奖励超越既往设计的候选，并在保持奖励时鼓励以更高效率实现超越。优化后，我们保守地更新支架库：候选只有在达到或超过当前奖励前沿，并严格改善至少一个前沿维度（奖励本身、延迟或成本）时才会保留。在训练期间，这些反馈同时更新 θ 和 $\mathcal{B}_n$ ；部署时， θ 保持冻结。

4.2 推理架构

JIT-Agent 支持两种推理模式：*静态推理*和*流式推理*；二者的区别在于任务完成后是丢弃经验，还是保留经验以支持后续任务。

静态推理。 在该模式下，我们引入一种轻量级测试时扩展方式：JIT-Agent 并行生成 N 个支架，从中选择一个，并且只执行选中的支架。这样可以在不增加环境回合数的情况下提高候选多样性。选中支架遵循前述相同的验证和有界修复流程。

流式推理。 流式推理模式旨在让有用经验沿任务序列向后传递。对于第 n 个任务 τ_n ，JIT-Agent 从当前支架库 $\mathcal{B}_n$ 中检索，生成并选择支架 $\mathbf{h}_n^\dagger$ ，然后执行一次。所得环境反馈仅用于判断该经验是否应更新支架库：

$$\xi_n \sim \text{Rollout}(\tau_n, \pi_\psi, \mathbf{h}_n^\dagger, \mathcal{C}_{\tau_n}; \Pi), \mathbf{m}_n = \text{Eval}(\xi_n), \quad (8)$$

$$\mathcal{B}_{n+1} = \text{Update}_{\text{III}}(\mathcal{B}_n; \tau_n, \mathbf{h}_n^\dagger, \mathbf{m}_n), \mathcal{E}_{\tau_{n+1}, n+1} = \text{Retrieve}(\tau_{n+1}; \mathcal{B}_{n+1}),$$

其中， $\mathbf{m}_n = (r_n, \bar{\ell}_n, \bar{\kappa}_n)$ 包含奖励、平均延迟和平均货币成本。按照阶段 III 的保留规则，当已完成的支架没有带来可接受的改进时， $\text{Update}_{\text{III}}$ 保持 $\mathcal{B}_n$ 不变；否则，保留的支架将成为后续任务的潜在参考。因此，流式推理通过不断演化的支架库迁移既往经验，既不会把环境反馈注入当前回合，也不会更新模型参数。

5 实验与分析

5.1 实验设置

评估基准。 我们在覆盖四类任务的九个基准上评估 JIT-Agent。**深度研究**包括 BrowseComp-Plus (Chen et al., 2025b) (准确率)、DeepSearchQA (Gupta et al., 2026) (答案 F1) 和 xBench-DeepSearch (xBench-DS) (Chen et al., 2025a) (准确率)；**日常工作**包括 AgentIF-Oneday (Chen et al., 2026a) (归一化加权评分规约得分) 和 PinchBench (PinchBench, 2026) (平均分)；**规划**包括 DeepPlanning-Shopping (购物车匹配率) 和 DeepPlanning-Travel (复合约束得分) 子集 (Zhang et al., 2026b)；**工作区**包括 OfficeBench (Wang et al., 2024) 和 OdysseyBench (Wang et al., 2025) (二者均使用任务成功率)。这些基准共同覆盖证据密集型搜索、长视野指令遵循、约束感知规划和跨应用工作区执行。所有指标均按 0–100 标度报告，越高越好。

骨干模型。 JIT-Agent 基于 Qwen3.6-27B 训练。我们主要将 JIT-Agent 与 GLM-5.2 (Z.ai, 2026) 和 DeepSeek-V4-Flash-Preview (DeepSeek-AI, 2026) 组合。这些骨干提供互补的工作点：GLM-5.2 是强大的开放长视野模型，而 DeepSeek-V4-Flash 强调推理效率。为检验泛化能力，我们还使用了 Qwen3.6 和 Momo-V2.5 系列中的其他模型。

表 2: 九个智能体基准上的主要结果。以 JIT-Agent 为前缀的行与对应原生模型使用相同骨干, 但把默认脚手架替换为 JIT 生成的支架。分数按 0-100 标度报告, 越高越好。每列的第一、第二和第三名分别用浅绿色、黄绿色和黄色标出; 第一名同时加粗。

模型	深度研究			日常工作		规划		工作区	
	BC+	DSQA	xBench	AgentIF	PinchBench	Shop	Travel	Office	Odyssey
Qwen3.7-Plus	70.5	78.0	75.0	59.9	80.5	75.2	52.1	56.6	67.7
GLM-5.2	72.0	89.2	76.0	63.0	87.0	78.2	62.8	63.0	75.3
DeepSeek-V4-Flash	68.1	76.2	70.1	58.4	81.7	59.1	54.8	61.0	71.0
DeepSeek-V4-Pro	71.4	72.4	79.0	56.5	61.1	71.1	55.2	62.0	72.0
Kimi K2.7 Code	73.6	87.8	79.0	57.3	76.1	72.8	56.9	65.8	69.9
GPT-5.6	76.9	76.0	81.0	68.0	84.2	83.7	84.9	65.3	68.7
Gemini 3.1 Pro	67.5	75.8	83.0	60.1	81.0	77.4	70.7	60.6	74.0
Gemini 3.5 Flash	75.0	88.0	85.0	64.0	74.2	76.2	50.3	63.3	78.0
JIT-Agent + GLM-5.2	78.0	93.9	88.0	69.9	93.3	83.4	83.0	68.4	78.7
JIT-Agent + DeepSeek-V4-Flash	74.0	85.1	82.0	63.8	92.9	83.9	61.3	63.4	73.0

基线。 我们采用两组互补基线。第一组为广泛比较, 包括原生 Qwen3.7-Plus (Qwen Team, 2026)、GLM-5.2、DeepSeek-V4-Flash/Pro (均为预览版)、Kimi K2.7 Code (Moonshot AI, 2026)、GPT-5.6-Sol (OpenAI, 2026), 以及 Gemini 3.1 Pro/3.5 Flash (Google DeepMind, 2026a;b)。第二组固定骨干模型, 将 JIT-Agent 与五种可复用智能体支架比较: CLAUDE CODE (Anthropics, 2025)、CODEX (OpenAI, 2025)、OPENCODE (OpenCode-AI, 2025)、HERMES (Nous Research, 2026) 和 NANO BOT (HKUDS, 2026)。第一组衡量端到端竞争力, 第二组则把支架设计的贡献与模型权重的贡献分离开来。

5.2 主要结果

表 2 在九个基准上比较了 JIT 生成的支架、原生骨干模型和前沿模型。替换默认脚手架后, 全部 18 个匹配的骨干-基准组合都有提升: GLM-5.2 的平均分从 74.1 升至 81.8 (+7.7), DeepSeek-V4-Flash 则从 66.7 升至 75.5 (+8.8)。最大增益出现在长视野规划任务: DeepSeek-V4-Flash 在 DeepPlanning-Shopping 上提高 24.8 分 (59.1→83.9), GLM-5.2 在 DeepPlanning-Travel 上提高 20.2 分 (62.8→83.0); DeepSeek-V4-Flash 在 xBench-DS 和 DeepSearchQA 上还分别提高 11.9 和 8.9 分。尽管采用开放骨干, 配备 JIT 的系统仍在九列中的八列领先: JIT-Agent + GLM-5.2 在七个基准排名第一, 包括 DeepSearchQA 的 93.9、AgentIF 的 69.9 和 PinchBench 的 93.3; JIT-Agent + DeepSeek-V4-Flash 则以 83.9 领先 DeepPlanning-Shopping, 并在所有已报告基准上超过 DeepSeek-V4-Pro, 平均优势为 8.7 分。唯一例外是 DeepPlanning-Travel, 配备 JIT 的 GLM-5.2 与 GPT-5.6 相差不超过 1.9 分。这些结果表明, 任务自适应支架会持续提升固定骨干的能力, 而非只对少数任务有效。

5.3 与先进智能体支架的比较

表 3 固定骨干模型, 在 DeepSearchQA、xBench-DS 和 AgentIF 上比较 DeepSeek-V4-Flash 与 Qwen3.6-Flash 搭配六种支架时的表现, 并报告性能、词元数和每样例 API 成本。JIT-Agent 在六种设置中的四种取得最佳性能: 使用 DeepSeek-V4-Flash 时, 它在 DeepSearchQA 和 xBench-DS 上分别超过最强固定支架 4.7 和 4.0 分; 使用 Qwen3.6-Flash 时, 则在 xBench-DS 和 AgentIF 上分别领先 7.0 和 2.9 分。在其余两种设置中, 它仅落后最佳分数 3.1 和 3.9 分, 却使用显著更少的词元, 说明不存在跨任务全面占优的固定支架。更重要的是, JIT-Agent 在全部六种设置中都具有最低的词元消耗和成本; 相较于成本最低的固定替代方案, 每样例成本下降 14.9-54.1%, 平均下降 36.0%。在 DeepSeek-V4-Flash 的 xBench-DS 上, 词元数从 527K 降至 212K, 成本从 \$0.075 降至 \$0.039, 同时性能由 78.0 提高到 82.0; 在 Qwen3.6-Flash 的 AgentIF 上, JIT-Agent 以 394K 词元和 \$0.078 成本达到 58.3, 而固定支架至少需要 839K 词元和 \$0.170。因此, 这些收益来自任务条件化脚手架, 而非更长的轨迹。

5.4 成本-性能 PARETO 前沿

图 3 展示了 DeepSearchQA 和 AgentIF 受控比较中的成本-性能几何关系。每个点都是表 3 中的一个骨干-支架组合; 越接近左上角越好。在 DeepSearchQA 上, DeepSeek-V4-Flash + JIT-Agent 是评估组合中一个很强的 Pareto 最优工作点: 每样例成本 \$0.066, 得分 85.1。相较于最强固定支架 NanoBot, 它提高 4.7 分, 同时降低 49.6% 的成本 (\$0.131→\$0.066)。对 Qwen3.6-Flash 而言, 生成支架提供了计算量更低的工作点: 它以 \$0.095 获得 70.3 分,

表 3: 与先进智能体支架的受控比较。对于每个固定骨干, 我们报告任务性能 (Perf.)、每个样例的平均词元消耗 (千词元, #Tokens (K)), 以及每个样例以美元计的 API 成本。性能越高越好, 词元数和成本越低越好。每个骨干组内的最优值以粗体标出。

骨干	支架	DeepSearchQA			xBench-DS			AgentIF		
		性能↑	词元 (K)	成本↓	性能↑	词元 (K)	成本↓	性能↑	词元 (K)	成本↓
DeepSeek-V4-Flash	CLAUDE CODE	79.6	625	\$0.088	75.0	559	\$0.079	66.9	808	\$0.114
	CODEX	77.8	760	\$0.107	70.0	680	\$0.096	58.5	870	\$0.123
	OPENCODE	75.9	1,832	\$0.258	65.0	1,157	\$0.159	48.1	950	\$0.135
	HERMES	69.9	1,157	\$0.163	72.0	1,254	\$0.177	60.3	1,000	\$0.142
	NANOBOT	80.4	924	\$0.131	78.0	527	\$0.075	53.1	1,034	\$0.147
	JIT-Agent	85.1	400	\$0.066	82.0	212	\$0.039	63.8	476	\$0.097
Qwen3.6-Flash	CLAUDE CODE	72.8	710	\$0.140	58.0	650	\$0.128	55.4	900	\$0.177
	CODEX	68.5	980	\$0.193	52.0	874	\$0.172	34.2	839	\$0.170
	OPENCODE	64.1	1,968	\$0.384	44.0	1,300	\$0.256	38.7	961	\$0.217
	HERMES	60.8	1,319	\$0.256	36.0	1,155	\$0.227	49.7	1,184	\$0.223
	NANOBOT	74.2	892	\$0.197	63.0	597	\$0.119	43.5	950	\$0.187
	JIT-Agent	70.3	464	\$0.095	70.0	300	\$0.069	58.3	394	\$0.078

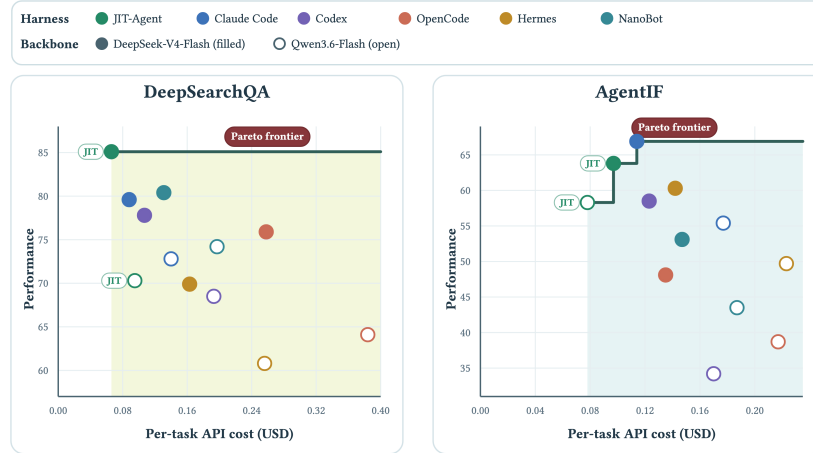


图 3: DeepSearchQA 与 AgentIF 上的成本-性能权衡。标记颜色表示支架, 实心圆与空心圆分别表示 DeepSeek-V4-Flash 和 Qwen3.6-Flash。横轴是每样例以美元计的 API 成本, 纵轴是任务性能。深绿色阶梯线描绘全局 Pareto 前沿; 浅黄绿色和蓝绿色区域中的组合至少被一个 Pareto 最优点支配。

而 NanoBot 以 \$0.197 获得 74.2 分, 即以 3.9 分的代价将成本降低 51.8%。AgentIF 呈现出另一条互补的前沿。使用 DeepSeek-V4-Flash 时, JIT-Agent 相较 Claude Code 将成本降低 14.9% (\$0.097 对 \$0.114), 同时性能差距不超过 3.1 分 (63.8 对 66.9), 因此二者都位于前沿上。使用 Qwen3.6-Flash 时, JIT-Agent 严格支配所有固定支架: 最佳分数从 55.4 提高到 58.3, 最低观测成本则从 \$0.170 降至 \$0.078。因此, 前沿依赖任务, 但 JIT 点持续左移说明任务自适应支架提高了推理效率, 而非通过延长轨迹来换取准确率。

5.5 跨模型对的泛化

为检验跨模型家族和规模的迁移能力, 我们在四个基准上, 以 DeepSeek V4、Qwen3.6 和 MIMO-V2.5 系列中的六个固定骨干比较 ReAct 与 JIT 生成支架。图 4 表明, JIT-Agent 在全部 24 个匹配比较中获胜, 平均领先 7.6 分; 三个家族的增益分别为 10.2、4.0 和 8.6 分。DeepSearchQA 受益最大 (平均 +15.2), 其中 MIMO-V2.5-Pro 提高 22.2 分, DeepSeek-V4-Flash 提高 19.0 分; DeepPlanning-Shopping 的增益则达到 24.8。这些在同一骨干内部持续出现的提升说明, 支架智能能够跨模型家族和变体迁移, 而非仅仅补偿某一个骨干。

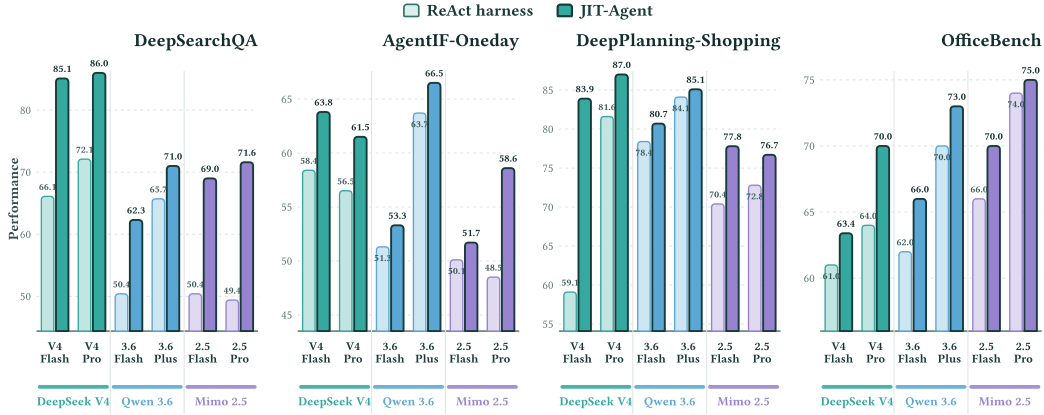


图 4: JIT 生成的支架相较 ReAct 持续改善配对骨干。四个面板分别报告 DeepSearchQA、AgentIF-Oneday、DeepPlanning-Shopping 和 OfficeBench。每个面板内的成对柱状条比较同一骨干在固定 ReAct 支架和 JIT 生成支架下的表现；六个骨干按模型家族分组，分数标注在柱状条上方。DeepSearchQA 使用 100 个样例的子集，其余三个基准使用 50 个样例的子集。

JIT HARNESS ATLAS · 01

Palimpsest — graph-planned artifact execution

A task becomes a dependency graph, executes with bounded parallelism, and leaves reusable state.

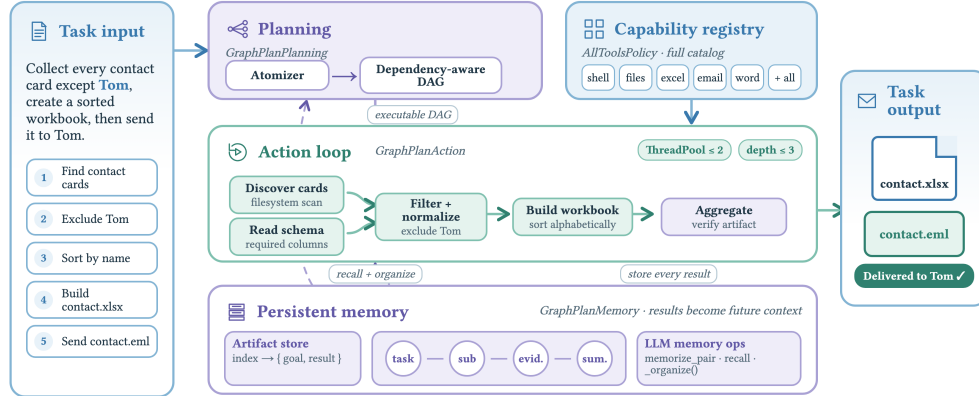


图 5: Palimpsest: 由图规划的工件执行。GraphPlanPlanning 将联系人处理请求转化为 DAG；GraphPlanAction 以有界宽度和深度执行该图，而 GraphPlanMemory 存储可复用工件与推理状态。

5.6 生成支架的可视化

为考察生成出的结构，图 5 展示了 PALIMPSEST：一种用于发现联系人卡片、筛选并规范化记录、构建工作簿并通过电子邮件交付的支架。它把这些依赖编译为 DAG：发现先于筛选，工作簿构建等待规范化记录，而交付发生在工件验证之后。GraphPlanAction 以有界宽度和深度执行就绪节点，GraphPlanMemory 则为下游节点保留目标、证据和工件。这种由任务塑造的规划、记忆与执行协调超越了提示变化，并把请求转化为可验证的生产流水线；节 C 还展示了递归委派、上下文折叠、工具门控、阶段条件化执行、证据跟踪、数值状态和工作区修复等互补结构。

6 结论

我们提出了 JIT-Agent：一种专门用于在推理时合成、修复和演化任务条件化智能体支架的模型。JIT-Agent 以可组合四模块协议表示支架，并通过定制学习、修复监督和 Evo-GDPO

训练生成器，从而把支架构建从人工设计的预先工件转变为学习所得的能力。在深度研究、日常工作、规划和工作区任务中，JIT 生成的支架持续增强其底层骨干，能够与前沿模型及先进固定支架竞争，并改善成本-性能权衡。这些结果确立了支架智能，即调整模型赖以运行的运行脚手架的能力，是一种超越模型权重本身、可训练且可迁移的智能体能力来源。

贡献者

核心贡献者：Guibin Zhang、Leo Lu、Fangzhou Xie

贡献者：Kang Zhu、Junhao Wang、Zhifei Xie、Zhaochen Yu、Zihang Liu、Zhongxiang Sun、Qiankun Li、Yue Liao、Heng Chang、Xiaobin Hu、Qibing Ren

通讯作者：Wangchunshu Zhou、Chuanrui Hu、Yafeng Deng、Shuicheng Yan

机构：新加坡国立大学、EverMind AI、南洋理工大学

参考文献

Salaheddin Alzu'bi, Baran Nama, Arda Kaz, Anushri Eswaran, Weiyuan Chen, Sarvesh Khetan, Rishab Bala, Tu Vu, and Sewoong Oh. Roma: Recursive open meta-agent framework for long-horizon multi-agent systems, 2026. URL <https://arxiv.org/abs/2602.01848>.

Anthropics. GitHub - anthropics/claude-code: Claude Code is an agentic coding tool that lives in your terminal, understands your codebase, and helps you code faster by executing routine tasks, explaining complex code, and handling git workflows - all through natural language commands. — github.com. <https://github.com/anthropics/claude-code>, 2025. [Accessed 04-01-2026].

Apodex Team. Apodex-1.0: Self-evolving heavy-duty solver. <https://www.apodex.com/blog/apodex-1.0>, 2026.

Fei Bai, Huatong Song, Shuang Sun, Daixuan Cheng, Yike Yang, Chuan Hao, Renyuan Li, Feng Chang, Yuan Wei, Ran Tao, Bryan Dai, Jian Yang, Wayne Xin Zhao, and Ji-Rong Wen. Clawgym: A scalable framework for building effective claw agents, 2026. URL <https://arxiv.org/abs/2604.26904>.

Pengfei Cao, Tianyi Men, Wencan Liu, Jingwen Zhang, Xuzhao Li, Xixun Lin, Dianbo Sui, Yanan Cao, Kang Liu, and Jun Zhao. Large language models for planning: A comprehensive and systematic survey, 2025. URL <https://arxiv.org/abs/2505.19683>.

Kaiyuan Chen, Yixin Ren, Yang Liu, Xiaobo Hu, Haotong Tian, Tianbao Xie, Fangfu Liu, Haoye Zhang, Hongzhang Liu, Yuan Gong, Chen Sun, Han Hou, Hui Yang, James Pan, Jianan Lou, Jiayi Mao, Jizheng Liu, Jinpeng Li, Kangyi Liu, Kenkun Liu, Rui Wang, Run Li, Tong Niu, Wenlong Zhang, Wenqi Yan, Xuancheng Wang, Yuchen Zhang, Yi-Hsin Hung, Yuan Jiang, Zexuan Liu, Zihan Yin, Zijian Ma, and Zhiwen Mo. xbench: Tracking agents productivity scaling with profession-aligned real-world evaluations, 2025a. URL <https://arxiv.org/abs/2506.13651>.

Kaiyuan Chen, Qimin Wu, Taiyu Hou, Tianhao Tang, Xueyu Hu, Yuchen Hou, Bikun Li, Chengming Qian, Guoyin Wang, Haolin Chen, Haotong Tian, Haoye Zhang, Haoyu Bian, Hongbing Pan, Hongkang Zhang, Hongyi Zhou, Jiaqi Cai, Jiewu Rao, Jiyan Ren, Keduan Huang, Lucia Zhu Huang, Mingyu Yuan, Naixu Guo, Qicheng Tang, Qinyan Zhang, Shuai Chen, Siheng Chen, Ting Ting Li, Xiaoxing Guo, Yaocheng Zuo, Yaoqi Guo, Yinan Wang, Yinzhou Yu, Yize Wang, Yuan Jiang, Yuan Tian, Yuanshuo Zhang, Yuxuan Liu, Yvette Yan Zeng, Zhenyu Shan, Zihan Yin, Xiaobo Hu, Yang Liu, Yixin Ren, and Yuan Gong. Agentifoneday: A task-level instruction-following benchmark for general ai agents in daily scenarios, 2026a. URL <https://arxiv.org/abs/2601.20613>.

Tingyang Chen, Shuo Lu, Kang Zhao, Weicheng Meng, Hanlin Teng, Tianhao Li, Chao Li, Xule Liu, Jian Liang, Zhizhong Zhang, Yuan Xie, Heng Qu, Kun Shao, and Jian Luan. Harnessx: A composable, adaptive, and evolvable agent harness foundry, 2026b. URL <https://arxiv.org/abs/2606.14249>.

- Zijian Chen, Xueguang Ma, Shengyao Zhuang, Ping Nie, Kai Zou, Andrew Liu, Joshua Green, Kshama Patel, Ruoxi Meng, Mingyi Su, Sahel Sharifymoghaddam, Yanxi Li, Hao-ran Hong, Xinyu Shi, Xuye Liu, Nandan Thakur, Crystina Zhang, Luyu Gao, Wenhui Chen, and Jimmy Lin. Browsecomp-plus: A more fair and transparent evaluation benchmark of deep-research agent, 2025b. URL <https://arxiv.org/abs/2508.06600>.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory, 2025. URL <https://arxiv.org/abs/2504.19413>.
- ClaudeCode. Agent Skills - Claude Code Docs — code.claude.com. <https://code.claude.com/docs/en/skills>, 2025. [Accessed 03-01-2026].
- DeepSeek-AI. Deepseek-v4 preview release. <https://api-docs.deepseek.com/news/news260424/>, 2026.
- Lufti Eren Erdogan, Nicholas Lee, Sehoon Kim, Suhong Moon, Hiroki Furuta, Gopala Anumanchipalli, Kurt Keutzer, and Amir Gholami. Plan-and-act: Improving planning of agents for long-horizon tasks. *arXiv preprint arXiv:2503.09572*, 2025a.
- Lufti Eren Erdogan, Nicholas Lee, Sehoon Kim, Suhong Moon, Hiroki Furuta, Gopala Anumanchipalli, Kurt Keutzer, and Amir Gholami. Plan-and-act: Improving planning of agents for long-horizon tasks, 2025b. URL <https://arxiv.org/abs/2503.09572>.
- Google DeepMind. Gemini 3.1 pro model card. <https://deepmind.google/models/model-cards/gemini-3-1-pro>, 2026a.
- Google DeepMind. Gemini 3.5 flash model card. <https://deepmind.google/models/model-cards/gemini-3-5-flash/>, 2026b.
- Nikita Gupta, Riju Chatterjee, Lukas Haas, Connie Tao, Andrew Wang, Chang Liu, Hidekazu Oiwa, Elena Gribovskaya, Jan Ackermann, John Blitzer, Sasha Goldshtein, and Dipanjan Das. Deepsearchqa: Bridging the comprehensiveness gap for deep research agents, 2026. URL <https://arxiv.org/abs/2601.20975>.
- HKUDS. Nanobot: Lightweight open-source ai agent. <https://github.com/HKUDS/nanobot>, 2026.
- Chuanrui Hu, Xingze Gao, Zuyi Zhou, Dannong Xu, Yi Bai, Xintong Li, Hui Zhang, Tong Li, Chong Zhang, Lidong Bing, and Yafeng Deng. Evermemos: A self-organizing memory operating system for structured long-horizon reasoning, 2026a. URL <https://arxiv.org/abs/2601.02163>.
- Mengkang Hu, Tianxing Chen, Qiguang Chen, Yao Mu, Wenqi Shao, and Ping Luo. Hiagent: Hierarchical working memory management for solving long-horizon agent tasks with large language model, 2024a. URL <https://arxiv.org/abs/2408.09559>.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024b.
- Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, Senjie Jin, Jiejun Tan, Yanbin Yin, Jiongnan Liu, Zeyu Zhang, Zhongxiang Sun, Yutao Zhu, Hao Sun, Boci Peng, Zhenrong Cheng, Xuanbo Fan, Jiabin Guo, Xinlei Yu, Zhenhong Zhou, Zewen Hu, Jiahao Huo, Junhao Wang, Yuwei Niu, Yu Wang, Zhenfei Yin, Xiaobin Hu, Yue Liao, Qiankun Li, Kun Wang, Wangchunshu Zhou, Yixin Liu, Dawei Cheng, Qi Zhang, Tao Gui, Shirui Pan, Yan Zhang, Philip Torr, Zhicheng Dou, Ji-Rong Wen, Xuanjing Huang, Yu-Gang Jiang, and Shuicheng Yan. Memory in the age of ai agents, 2025. URL <https://arxiv.org/abs/2512.13564>.
- Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, Senjie Jin, Jiejun Tan, Yanbin Yin, Jiongnan Liu, Zeyu Zhang, Zhongxiang Sun, Yutao Zhu, Hao Sun, Boci Peng, Zhenrong Cheng, Xuanbo Fan, Jiabin Guo, Xinlei Yu, Zhenhong Zhou, Zewen Hu, Jiahao Huo, Junhao Wang, Yuwei

- Niu, Yu Wang, Zhenfei Yin, Xiaobin Hu, Yue Liao, Qiankun Li, Kun Wang, Wangchunshu Zhou, Yixin Liu, Dawei Cheng, Qi Zhang, Tao Gui, Shirui Pan, Yan Zhang, Philip Torr, Zhicheng Dou, Ji-Rong Wen, Xuanjing Huang, Yu-Gang Jiang, and Shuicheng Yan. Memory in the age of ai agents, 2026b. URL <https://arxiv.org/abs/2512.13564>.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, et al. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.
- LangGraph. GitHub - langchain-ai/langgraph: Build resilient language agents as graphs. — github.com. <https://github.com/langchain-ai/langgraph>, 2025. [Accessed 03-01-2026].
- Hyunin Lee, Jinglue Xu, Jeffrey Seely, Donghyun Lee, Matei Zaharia, and Yujin Tang. Recursive harness self-improvement, 2026a. URL <https://arxiv.org/abs/2607.15524>.
- Sanghoon Lee, Jiyeong Chae, and Kyung-Joon Park. Harness engineering for physical ai: Robot middleware is the harness layer, 2026b. URL <https://arxiv.org/abs/2606.09416>.
- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses, 2026c. URL <https://arxiv.org/abs/2603.28052>.
- Yoonsang Lee, Howard Yen, Xi Ye, and Danqi Chen. Agentic aggregation for parallel scaling of long-horizon agentic tasks, 2026d. URL <https://arxiv.org/abs/2604.11753>.
- Xiaoxi Li, Wenxiang Jiao, Jiarui Jin, Guanting Dong, Jiajie Jin, Yinuo Wang, Hao Wang, Yutao Zhu, Ji-Rong Wen, Yuan Lu, and Zhicheng Dou. Deepagent: A general reasoning agent with scalable toolsets, 2026. URL <https://arxiv.org/abs/2510.21618>.
- Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses, 2026. URL <https://arxiv.org/abs/2604.25850>.
- Shih-Yang Liu, Xin Dong, Ximing Lu, Shizhe Diao, Peter Belcak, Mingjie Liu, Min-Hung Chen, Hongxu Yin, Yu-Chiang Frank Wang, Kwang-Ting Cheng, Yejin Choi, Jan Kautz, and Pavlo Molchanov. Gdpo: Group reward-decoupled normalization policy optimization for multi-reward rl optimization, 2026a. URL <https://arxiv.org/abs/2601.05242>.
- Zewen Liu, Zhan Shi, Yisi Sang, Bing He, Minhua Lin, Tianxin Wei, Dakuo Wang, Benoit Dumoulin, Wei Jin, and Hanqing Lu. Adaptive auto-harness: Sustained self-improvement for agentic system deployment on open-ended task streams, 2026b. URL <https://arxiv.org/abs/2606.01770>.
- Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P. Murphy. AutoHarness: Improving llm agents by automatically synthesizing a code harness, 2026. URL <https://arxiv.org/abs/2603.03329>.
- Junyu Luo, Weizhi Zhang, Ye Yuan, Yusheng Zhao, Junwei Yang, Yiyang Gu, Bohan Wu, Binqi Chen, Ziyue Qiao, Qingqing Long, Rongcheng Tu, Xiao Luo, Wei Ju, Zhiping Xiao, Yifan Wang, Meng Xiao, Chenwu Liu, Jingyang Yuan, Shichang Zhang, Yiqiao Jin, Fan Zhang, Xian Wu, Hanqing Zhao, Dacheng Tao, Philip S. Yu, and Ming Zhang. Large language model agent: A survey on methodology, applications and challenges, 2025. URL <https://arxiv.org/abs/2503.21460>.
- Qianyu Meng, Yanan Wang, Liyi Chen, Qimeng Wang, Chengqiang Lu, Wei Wu, Yan Gao, Yi Wu, and Yao Hu. Agent harness for large language model agents: A survey. *Preprints*, April 2026a. doi: 10.20944/preprints202604.0428.v2. URL <https://doi.org/10.20944/preprints202604.0428.v2>.
- Qianyu Meng, Yanan Wang, Liyi Chen, Qimeng Wang, Chengqiang Lu, Wei Wu, Yan Gao, Yi Wu, and Yao Hu. Agent harness for large language model agents: A survey. 2026b.

- Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Darius Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL <https://arxiv.org/abs/2601.11868>.
- Microsoft. GitHub - ag2ai/ag2: AG2 (formerly AutoGen): The Open-Source AgentOS. <https://github.com/ag2ai/ag2>, 2024. [Accessed 31-08-2025].
- Moonshot AI. Kimi k2.7 code: Open-source agentic coding model. <https://www.kimi.com/resources/kimi-k2-7-code>, 2026.
- Jun Nie, Yonggang Zhang, Jun Song, Qianshu Cai, Dahai Yu, Yike Guo, Xinmei Tian, and Bo Han. TTHe: Test-time harness evolution, 2026. URL <https://arxiv.org/abs/2607.08124>.
- Xuying Ning, Katherine Tieu, Dongqi Fu, Tianxin Wei, Zihao Li, Yuanchen Bei, Jiaru Zou, Mengting Ai, Zhining Liu, Ting-Wei Li, et al. Code as agent harness. *arXiv preprint arXiv:2605.18747*, 2026.
- Nous Research. Hermes agent. <https://github.com/NousResearch/hermes-agent>, 2026.
- OpenAI. Codex | AI Coding Partner from OpenAI — openai.com. <https://openai.com/codex/>, 2025.
- OpenAI. Gpt-5.6: Frontier intelligence that scales with your ambition. <https://openai.com/index/gpt-5-6/>, 2026.
- OpenClaw Contributors. Openclaw: Your own personal ai assistant. <https://github.com/openclaw/openclaw>, 2026.
- OpenCode-AI. GitHub - opencode-ai/opencode: A powerful AI coding agent. Built for the terminal. — github.com. <https://github.com/opencode-ai/opencode>, 2025.
- Linyue Pan, Lexiao Zou, Shuo Guo, Jingchen Ni, and Hai-Tao Zheng. Natural-language agent harnesses. *arXiv preprint arXiv:2603.25723*, 2026.
- PinchBench. Best Models by Success Rate | PinchBench — pinchbench.com. <https://pinchbench.com/>, 2026. [Accessed 11-04-2026].
- Hongjin Qian, Zhao Cao, and Zheng Liu. Memobrain: Executive memory as an agentic brain for reasoning, 2026. URL <https://arxiv.org/abs/2601.08079>.
- Tianrui Qin, Qianben Chen, Sinuo Wang, He Xing, King Zhu, He Zhu, Dingfeng Shi, Xinxin Liu, Ge Zhang, Jiaheng Liu, Yuchen Eleanor Jiang, Xitong Gao, and Wangchunshu Zhou. Flash-searcher: Fast and effective web agents via dag-based parallel execution, 2025. URL <https://arxiv.org/abs/2509.25301>.
- Qwen Team. Qwen3.7-plus: Multimodal agent intelligence. <https://qwen.ai/home>, 2026.

- Jianhao Ruan, Zhihao Xu, Yiran Peng, Fashen Ren, Zhaoyang Yu, Xinbing Liang, Jinyu Xiang, Yongru Chen, Bang Liu, Chenglin Wu, Yuyu Luo, and Jiayi Zhang. Aorchestra: Automating sub-agent creation for agentic orchestration, 2026. URL <https://arxiv.org/abs/2602.03786>.
- Anjana Sarkar and Soumyendu Sarkar. Survey of llm agent communication with mcp: A software design pattern centric review, 2025. URL <https://arxiv.org/abs/2506.05364>.
- Yu Shang, Yu Li, Keyu Zhao, Likai Ma, Jiahe Liu, Fengli Xu, and Yong Li. Agentsquare: Automatic llm agent search in modular design space. *arXiv preprint arXiv:2410.06153*, 2024.
- Shuai Shao, Kangning Zhang, Qingyao Li, Shijian Wang, Hao Wang, Wenxiang Jiao, Yuan Lu, Yi Guo, Weiwen Liu, and Weinan Zhang. Harness-R1: Learning to edit executable runtime harnesses from agent failure trajectories, 2026. URL <https://arxiv.org/abs/2608.02276>.
- Dingfeng Shi, Jingyi Cao, Qianben Chen, Weichen Sun, Weizhen Li, Hongxuan Lu, Fangchen Dong, Tianrui Qin, King Zhu, Minghao Liu, Jian Yang, Ge Zhang, Jiaheng Liu, Changwang Zhang, Jun Wang, Yuchen Eleanor Jiang, and Wangchunshu Zhou. Taskcraft: Automated generation of agentic tasks, 2025. URL <https://arxiv.org/abs/2506.10055>.
- Xiaoshuai Song, Haofei Chang, Guanting Dong, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. Envscaler: Scaling tool-interactive environments for llm agent via programmatic synthesis, 2026. URL <https://arxiv.org/abs/2601.05808>.
- Weixuan Wang, Dongge Han, Daniel Madrigal Diaz, Jin Xu, Victor Rühle, and Saravan Rajmohan. Odysseybench: Evaluating llm agents on long-horizon complex office application workflows, 2025. URL <https://arxiv.org/abs/2508.09124>.
- Zilong Wang, Yuedong Cui, Li Zhong, Zimin Zhang, Da Yin, Bill Yuchen Lin, and Jingbo Shang. Officebench: Benchmarking language agents across multiple applications for office automation, 2024. URL <https://arxiv.org/abs/2407.19056>.
- Jialong Wu, Wenbiao Yin, Yong Jiang, Zhenglin Wang, Zekun Xi, Runnan Fang, Linhai Zhang, Yulan He, Deyu Zhou, Pengjun Xie, and Fei Huang. Webwalker: Benchmarking llms in web traversal, 2025. URL <https://arxiv.org/abs/2501.07572>.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation framework, August 01, 2023 2023.
- Xixi Wu, Kuan Li, Yida Zhao, Liwen Zhang, Litu Ou, Huifeng Yin, Zhongwang Zhang, Xinmiao Yu, Dingchu Zhang, Yong Jiang, Pengjun Xie, Fei Huang, Minhao Cheng, Shuai Wang, Hong Cheng, and Jingren Zhou. Resum: Unlocking long-horizon search intelligence via context summarization, 2026. URL <https://arxiv.org/abs/2509.13313>.
- Renjun Xu and Yang Yan. Agent skills for large language models: Architecture, acquisition, security, and the path forward, 2026. URL <https://arxiv.org/abs/2602.12430>.
- B. Y. Yan, Chaofan Li, Hongjin Qian, Shuqi Lu, and Zheng Liu. General agentic memory via deep research, 2025. URL <https://arxiv.org/abs/2511.18423>.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024. URL <https://arxiv.org/abs/2405.15793>.
- Xiaofang Yang, Lijun Li, Heng Zhou, Tong Zhu, Xiaoye Qu, Yuchen Fan, Qianshan Wei, Rui Ye, Li Kang, Yiran Qin, Zhiqiang Kou, Daizong Liu, Qi Li, Ning Ding, Siheng Chen, and Jing Shao. Toward efficient agents: Memory, tool learning, and planning, 2026. URL <https://arxiv.org/abs/2601.14192>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023a. URL <https://arxiv.org/abs/2210.03629>.

- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023b.
- Rui Ye, Zhongwang Zhang, Kuan Li, Huifeng Yin, Zhengwei Tao, Yida Zhao, Liangcai Su, Liwen Zhang, Zile Qiao, Xinyu Wang, Pengjun Xie, Fei Huang, Siheng Chen, Jingren Zhou, and Yong Jiang. Agentfold: Long-horizon web agents with proactive context management, 2025. URL <https://arxiv.org/abs/2510.24699>.
- Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. Textgrad: Automatic "differentiation" via text, 2024. URL <https://arxiv.org/abs/2406.07496>.
- Z.ai. Glm-5.2: Built for long-horizon tasks. <https://z.ai/blog/glm-5.2>, 2026.
- Alex L. Zhang, Tim Kraska, and Omar Khattab. Recursive language models, 2026a. URL <https://arxiv.org/abs/2512.24601>.
- Guibin Zhang, Haotian Ren, Chong Zhan, Zhenhong Zhou, Junhao Wang, He Zhu, Wangchunshu Zhou, and Shuicheng Yan. Memevolve: Meta-evolution of agent memory systems, 2025. URL <https://arxiv.org/abs/2512.18746>.
- Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. AFlow: Automating Agentic Workflow Generation, October 2024. URL <http://arxiv.org/abs/2410.10762>. arXiv:2410.10762.
- Yinger Zhang, Shutong Jiang, Renhao Li, Jianhong Tu, Yang Su, Lianghao Deng, Xudong Guo, Chenxu Lv, and Junyang Lin. Deepplanning: Benchmarking long-horizon agentic planning with verifiable constraints, 2026b. URL <https://arxiv.org/abs/2601.18137>.
- Yixian Zhang, Huanming Zhang, Feng Gao, Xiao Li, Zhihao Liu, Chunyang Zhu, Jiaxing Qiu, Yuchen Yan, Jiyuan Liu, Wenhao Tang, Zhengru Fang, Yi Nie, Changxu Wei, Yu Wang, Wenbo Ding, and Chao Yu. Harness vla: Steering frozen vlas into reliable manipulation primitives via memory-guided agents, 2026c. URL <https://arxiv.org/abs/2607.08448>.
- Chenyu Zhou, Huacan Chai, Wenteng Chen, Zihan Guo, Rong Shan, Yuanyi Song, Tianyi Xu, Yingxuan Yang, Aofan Yu, Weiming Zhang, Congming Zheng, Jiachen Zhu, Zeyu Zheng, Zhuosheng Zhang, Xingyu Lou, Changwang Zhang, Zhihui Fu, Jun Wang, Weiwen Liu, Jianghao Lin, and Weinan Zhang. Externalization in llm agents: A unified review of memory, skills, protocols and harness engineering, 2026. URL <https://arxiv.org/abs/2604.08224>.
- Tailin Zhou. Hierarchical self-improvement: A framework for task-specific evolvable agent harnesses, 2026. URL <https://arxiv.org/abs/2608.08466>.
- He Zhu, Tianrui Qin, King Zhu, Heyuan Huang, Yeyi Guan, Jinxiang Xia, Yi Yao, Hanhao Li, Ningning Wang, Pai Liu, Tianhao Peng, Xin Gui, Xiaowan Li, Yuhui Liu, Yuchen Eleanor Jiang, Jun Wang, Changwang Zhang, Xiangru Tang, Ge Zhang, Jian Yang, Minghao Liu, Xitong Gao, Jiaheng Liu, and Wangchunshu Zhou. Oagents: An empirical study of building effective agents, 2025. URL <https://arxiv.org/abs/2506.15741>.

A 支架优化范式

表 4 按支架的构建时机以及改进过程中的哪些部分由学习获得，对代表性支架优化方法进行组织。现有方法主要在部署前搜索耐久支架，或在线编辑该工件；JIT-Agent 则把构成本摊销到训练所得的生成器中，由其合成实例专用支架，并同时学习修复与演化。该比较针对结构，而非经验性能排名；它阐明了哪些能力是优化过程原生具备的，哪些仍属于外部搜索操作。

表 4: **支架优化范式**。构建方式区分预先搜索得到的支架、随后由测试时反馈编辑的预先支架，以及为每项任务即时生成的支架。实例合成、支架模型、学习式修复和在线演化分别表示一种方法是否直接合成实例专用支架、训练生成器、从失败执行轨迹中学习修复，以及在部署后持续改进。

方法	构建方式	实例合成	支架模型	学习式修复	在线演化
AutoHarness (Lou et al., 2026)	AOT (搜索)	✗	✗	✗	✗
Meta-Harness (Lee et al., 2026c)	AOT (搜索)	✗	✗	✗	✗
AHE (Lin et al., 2026)	AOT (搜索)	✗	✗	✗	✗
Adaptive AH (Liu et al., 2026b)	AOT (测试时编辑)	✗	✗	✗	✓
TTHE (Nie et al., 2026)	AOT (测试时编辑)	✗	✗	✗	✓
RHI (Lee et al., 2026a)	AOT (测试时编辑)	✗	✗	✗	✓
Harness-R1 (Shao et al., 2026)	AOT (测试时编辑)	✗	✓	✓	✓
JIT-Agent (本文)	JIT	✓	✓	✓	✓

B 更多结果

本节以完整的流式评估补充正文中的汇总比较，检验已完成的任务能否在不改变生成器参数的情况下改善支架库，从而帮助后续任务。

B.1 测试时支架演化

JIT-Agent 不仅用于一次性生成支架，还会依据执行反馈改善其档案。我们比较了支架生成彼此独立的静态 JIT，以及在整个评估流中持续检索并更新支架的流式 JIT。

图 6 表明，在 DeepPlanning-Shopping、DeepPlanning-Travel 和 OfficeBench 上，流式 JIT 的最终结果都高于静态变体。在三条任务流中，累积准确率优势都会随着执行反馈累积而显现，并一直保持到评估结束。相应的成本与工具调用轨迹说明，终点收益并不总是与更大的交互预算绑定。这一模式与 Evo-GDPO 的目标一致：只有能推进档案前沿的支架才会被保留。

C 更多生成支架的可视化

本附录以相同四模块协议生成的另外九个支架，补充节 5.6 中的代表性案例。这些示例涵盖有界递归研究、约束密集的购物与旅行、分阶段 Web 生产、线索驱动研究、证据敏感检索、数值分析和工作区操作。它们并非单一执行模板的变体：JIT-Agent 会选择适合每项请求结构的状态表示和控制机制，再围绕该选择实例化规划、动作、记忆和能力编排。总体而言，这些案例在执行语义而非提示措辞层面直观呈现了任务条件化的支架定制。

把深度研究表示为有界递归委派。 TRAPDOOR 处理一个多跳身份问题，其线索横跨一段历史引文、一次大学合并、一本回忆录和一篇研究论文。在此，固定 DAG 会过早锁定尚不确定的证据路径，因此生成的支架使用 DynamicDecomposer 形成并修订子问题，同时为研究工具策略加入合成的 delegate 能力。一次 delegate 调用会启动私有研究子智能体；它拥有独立记忆、仅用于研究的工具和五步预算，返回答案随后作为普通观察重新进入父循环。与此同时，FactGraphMemory 从不断增长的历史中提取紧凑的键-值事实，使父智能体能够跨分支整合证据，而不必继承每个子智能体的全部轨迹。因此，该支架把递归研究变成专门面向这类任务的运行时原语；任务中的不确定性通过分支式证据收集解决。



图 6: 跨任务流的流式测试时支架演化。图中展示 DeepPlanning-Shopping、DeepPlanning-Travel 和 OfficeBench 上的累积准确率（上）、每任务 API 成本（中）和每任务工具调用数（下）。虚线表示任务专用支架彼此独立生成的静态 JIT；实线表示在新任务到来时持续吸收执行反馈的流式 JIT。阴影区域表示流式轨迹周围的局部波动。流式 JIT 在三个基准上最终都取得更高的累积准确率，而 API 成本和工具使用轨迹仍依赖任务，且整体处于相近量级。

JIT HARNESS ATLAS · 02

Trapdoor — delegation as a runtime primitive

A synthetic tool call opens a bounded sub-agent; its answer returns through the ordinary observation loop.

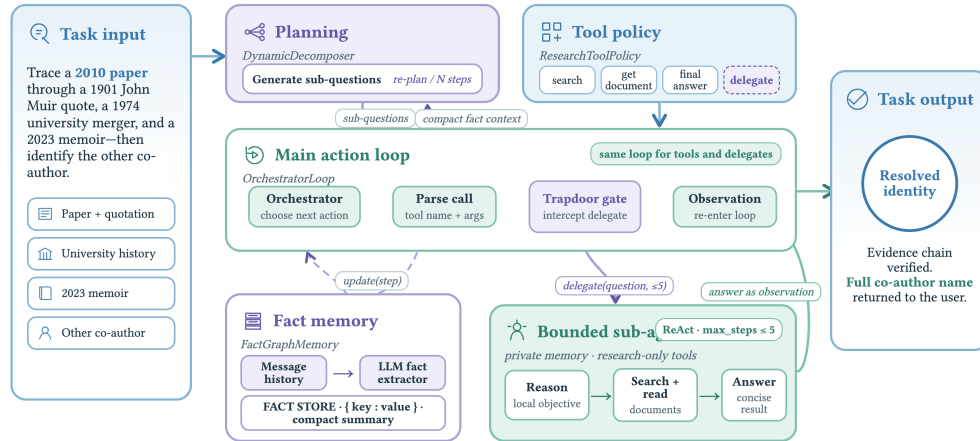


图 C.1: Trapdoor: 委派工具调用背后的有界研究。合成的 delegate 能力由 OrchestratorLoop 截获；后者运行有界子智能体，并把提取的事实写入 FactGraphMemory。

图 C.2和C.3 展示了两种通过记忆控制长视野执行的任务专用方式。ORIGAMI 必须在库存、配送、尺码、评论数和评分等耦合约束下组合一套季节性衣橱。ROMAPanning 把请求分解为相互依赖的商品搜索，HierarchicalAction 以有界深度并行运行至多两个分支，每个分支保留自己的轨迹和工件。当某个分支变长时，fold_thought 只替换其当前工作上下文；

JIT HARNESS ATLAS · 03

Origami — hierarchical work, folded context

Independent subtask memories preserve the work; a deliberate fold replaces only the active context.

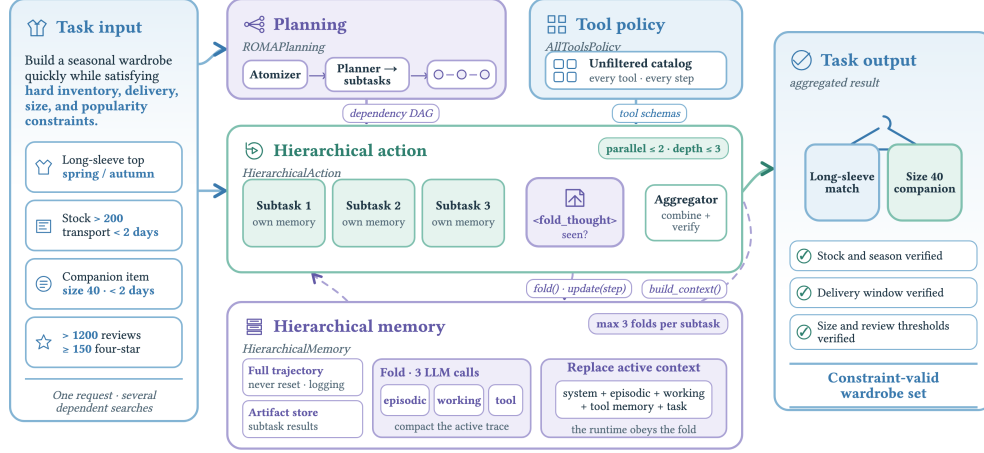


图 C.2: Origami: 带折叠上下文的层次化工作。ROMAPPlanning 创建相互隔离的子任务; HierarchicalMemory 保留其轨迹与工件, 而 fold_thought 只在聚合前替换当前工作上下文。

JIT HARNESS ATLAS · 04

Turnstile — evidence first, itinerary second

A task-specific checklist withholds final_answer until every required travel fact has been collected.

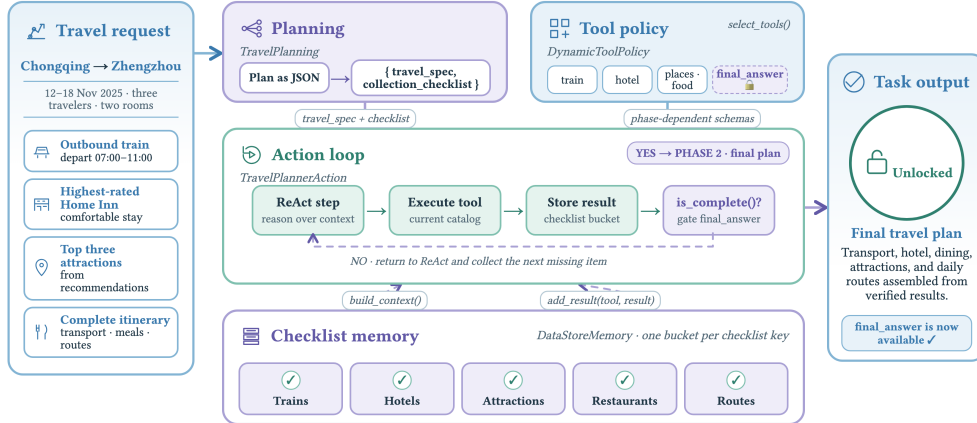


图 C.3: Turnstile: 先证据, 后行程。TravelPlanning 产生旅行规范和检查清单, DataStoreMemory 跟踪所需证据桶; 只有 is_complete() 成功后, DynamicToolPolicy 才暴露 final_answer。

已完成子任务的结果仍可用于最终约束检查。因此，该支架把上下文预算花在组合式购物任务尚未解决的部分，同时不丢弃先前的商品证据。

相比之下，TURNSTILE 服务于一项旅行请求，其中规定了日期、乘客、房间、出发时间窗口、评分最高的酒店、景点、餐饮和每日路线。生成的 TravelPlanning 模块把这些义务编译为带类型的 travel_spec 和收集清单，DataStoreMemory 则为每类证据分配专用桶。DynamicToolPolicy 为缺失的证据桶暴露搜索工具，但在 is_complete() 成功前不会暴露 final_answer。此处记忆的首要用途不是压缩长轨迹；它充当可执行覆盖契约，防止系统在收齐所需旅行事实之前合成行程。

JIT HARNESS ATLAS · 05

Gearbox — one phase shift rewires the agent

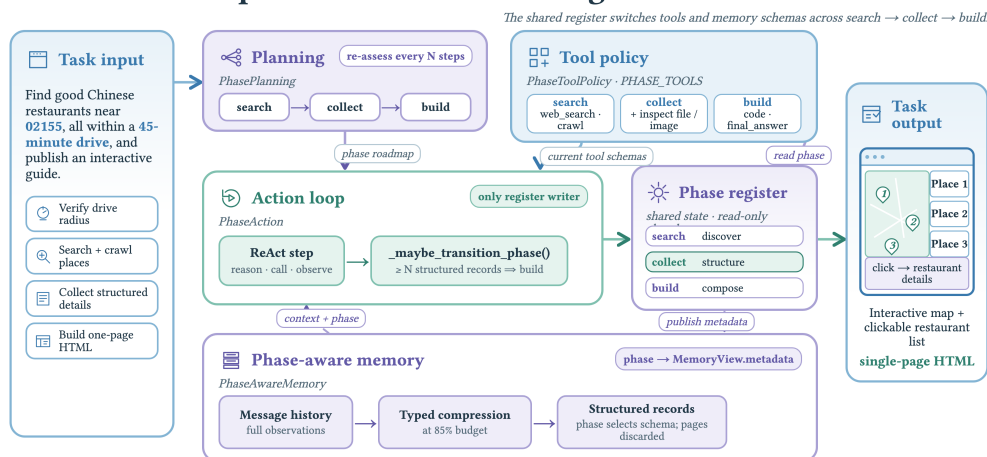


图 C.4: Gearbox：一次阶段切换会改写智能体。PhaseAction 是共享阶段寄存器的唯一写入者；PhaseToolPolicy 和 PhaseAwareMemory 读取该状态，从而同时切换暴露的能力与带类型记忆模式。

图 C.4和C.5 展示了显式状态如何持续重塑后续决策。GEARBOX 用于寻找指定行车半径内的中餐厅，并把结果发布为交互式单页指南。其 PhaseAction 是共享搜索-收集-构建寄存器的唯一写入者；只有收集到足够多的结构化餐厅记录后，系统才会转入构建阶段。PhaseToolPolicy 和 PhaseAwareMemory 都读取该寄存器，从 Web 发现切换到记录检查，再切换到编码和发布，同时把观察压缩为当前阶段所需的模式。因此，模型会创建一个阶段机，因为任务的模式会从证据获取转向工件构建。

PEGBBOARD 则处理一个由六条传记与电影线索定义的身份识别问题。它把进度表示为候选者 × 线索矩阵，其中每条提取出的主张都保留其文档标识符。空单元格和矛盾会驱动 DynamicResearchPlanning 开展定向搜索，而得到充分支持的行则触发独立验证阶段，重新打开引用文档并解决冲突。与通用研究记录不同，该状态使覆盖率和来源可追溯性能直接驱动动作。它与 GEARBOX 的对比说明，生成状态可以是小型阶段变量，也可以是由任务塑造的证据结构，具体取决于接下来必须由什么来支配动作。

图 C.6和C.7 对后续推理从先前执行中接收的内容进行了定制。APPRAISER 通过协调植物学特征与文本证据，回答一个古典本草身份识别问题。每完成一次搜索或阅读步骤，辅助评分器都会为观察赋予证据价值；EvidenceGuidedMemory 保留完整轨迹以供审计，但只把最早、最新和得分最高的 K 项观察渲染进下一轮上下文。因此，低价值探索不再与最有助于识别的段落竞争，同时持久历史中的证据不会被删除。该支架专门适配一类检索任务：其主要瓶颈是证据显著性而非工具选择。

ABACUS 面向一项数值分析请求生成：先比较地区平均值，再对获胜地区中的国家排名。该支架有意把聚合移出语言模型推理：StructuredReActAction 调用 shell 或 Python，从标准输出中提取带标记的 RESULT_JSON 载荷，必要时修复其语法，再把数值写入 StructuredStateMemory。随后，地区、国家、地区均值和国家均值都作为带类型状态

JIT HARNESS ATLAS · 06

Pegboard — research as an evidence matrix

Every observation fills a candidate × clue cell; coverage drives replanning and verification.

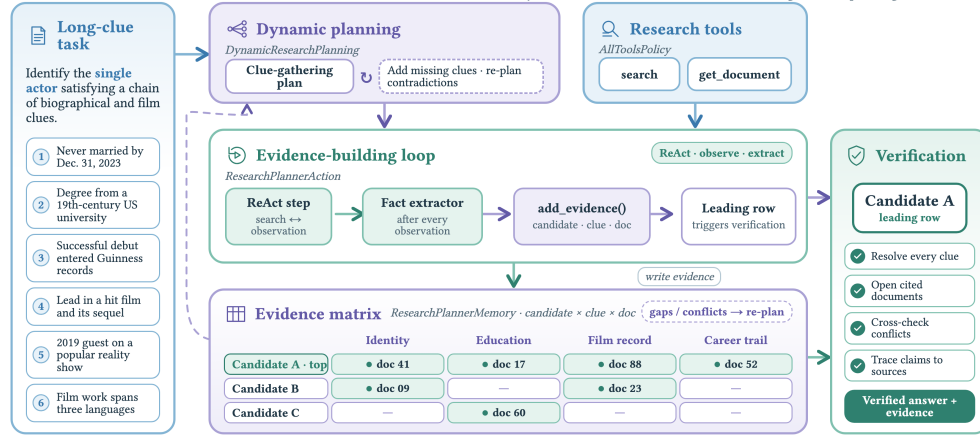


图 C.5: Pegboard: 把研究表示为证据矩阵。每项观察都连同文档标识符提取到候选者 × 线索单元格中; 矩阵覆盖率同时驱动 DynamicResearchPlanning 和向基于来源的验证阶段迁移。

JIT HARNESS ATLAS · 07

Appraiser — evidence decides what the model sees

Keep the full trajectory; render only the first, last, and highest-scoring evidence into the next context.

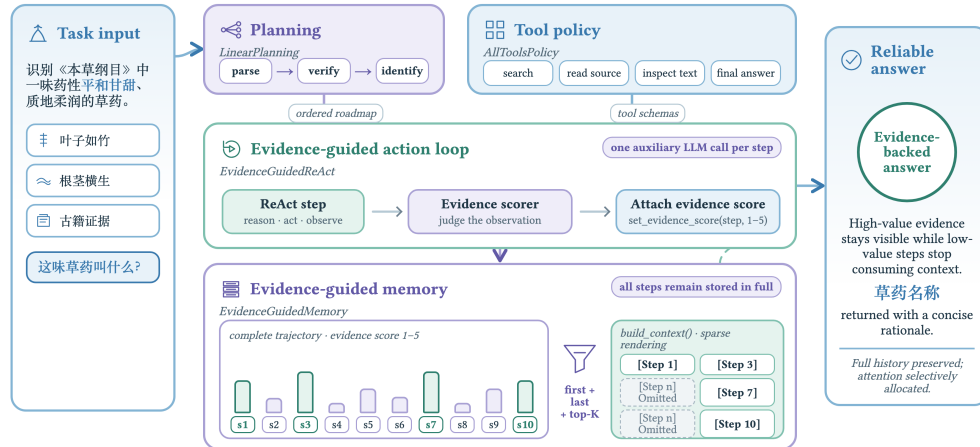


图 C.6: Appraiser: 证据决定模型看见什么。EvidenceGuidedReAct 为每一步评分; EvidenceGuidedMemory 存储完整运行过程, 但只把最早、最新和得分最高的 K 项观察渲染进 build_context()。

JIT HARNESS ATLAS · 08

Abacus — from computation to reliable typed state

Shell-derived evidence is parsed once, preserved structurally, and reported without re-deriving the numbers.

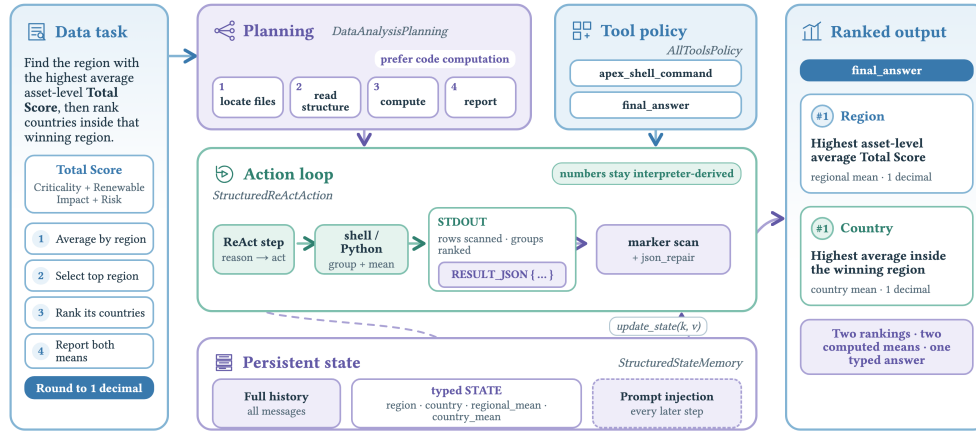


图 C.7: **Abacus: 从计算转向可靠的带类型状态。** StructuredReActAction 执行 shell 或 Python 代码，从标准输出中提取 RESULT_JSON 载荷，并更新 StructuredStateMemory，无需模型重新推导计算值。

注入每个后续步骤，使最终答案报告由解释器算出的数值，而非从文本中重新推导。两个案例都保留完整审计轨迹，但会围绕各自任务的主导可靠性要求渲染工作视图：前者关注证据选择，后者关注数值保真度。

JIT HARNESS ATLAS · 09

Player Piano — typed batch editing, verified file by file

A Python interpreter executes a structured plan; deterministic checks control every transition.

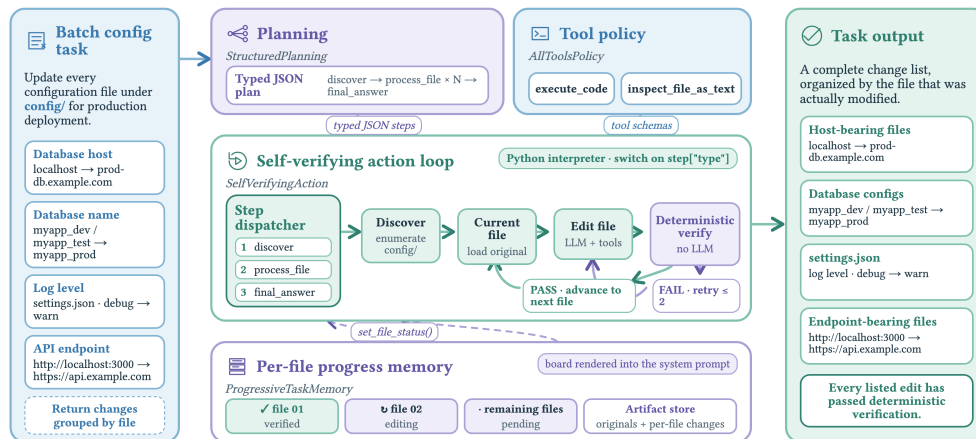


图 C.8: **Player Piano: 带类型的批量编辑，逐文件验证。** Python 调度器解释 StructuredPlanning 步骤，SelfVerifyingAction 无需大语言模型即可检查每次编辑，而 ProgressiveTaskMemory 维护逐文件看板和工件存储。

图 C.8和C.9 通过控制动作何时进入持久历史，为不同规模的工作区执行提供专用机制。PLAYER PIANO 处理配置文件的批量迁移，包括数据库、日志和 API 端点变更。StructuredPlanning 产生带类型的发现、处理文件和最终答案步骤，由 Python 调度器执行；

JIT HARNESS ATLAS · 10

Mulligan — repair the step, preserve the history

A failed action is regenerated in place; only successful repairs enter the transcript.

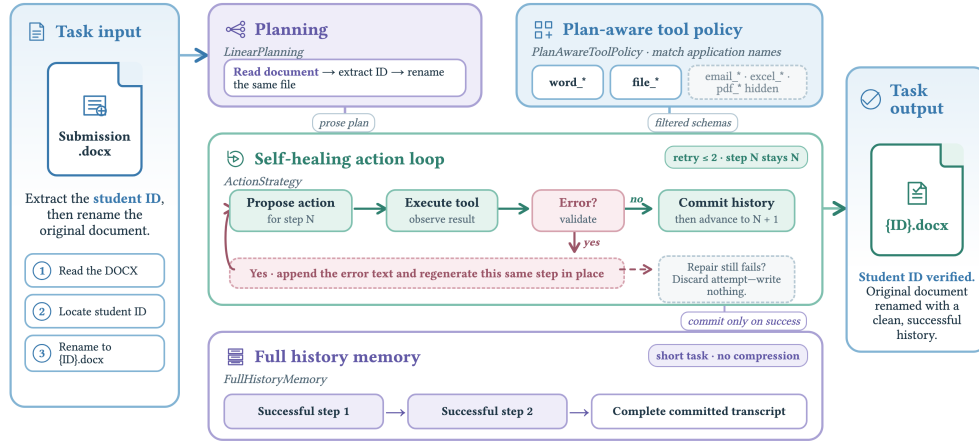


图 C.9: Mulligan: 修复步骤, 保留历史。PlanAwareToolPolicy 只暴露与计划相关的文件和文档工具; 失败动作会在同一步骤重新生成, 而 FullHistoryMemory 只接收成功执行。

每次编辑后, 一个确定性的非大语言模型检查决定前进还是重试。ProgressiveTaskMemory 把逐文件看板渲染进系统提示, 并同时保留原始文件与已接受的变更, 防止系统在大批量任务中忘记哪些文件真正通过了验证。

MULLIGAN 面向一个较短的文档任务生成: 从 DOCX 文件中提取学生标识符, 并重命名一个文件。其线性计划不需要层次结构或上下文压缩, 因此支架保留完整的成功历史, 只暴露 Word 和文件操作, 同时隐藏无关的电子邮件、电子表格和 PDF 工具。如果动作失败, 循环会在本地追加错误, 并在同一步骤重新生成, 最多尝试两次; 只有成功动作会在计划推进前提交。因此, 两种支架都关注可靠的工作区修改, 但实现方式不同: 一个使用带类型的批次进度和逐文件验证, 另一个使用窄工具暴露与事务式步骤修复。这种依赖任务的差异正是 JIT-Agent 所追求的行为: 可靠性机制根据请求的结构和失败面生成, 而非作为统一脚手架强加给所有任务。